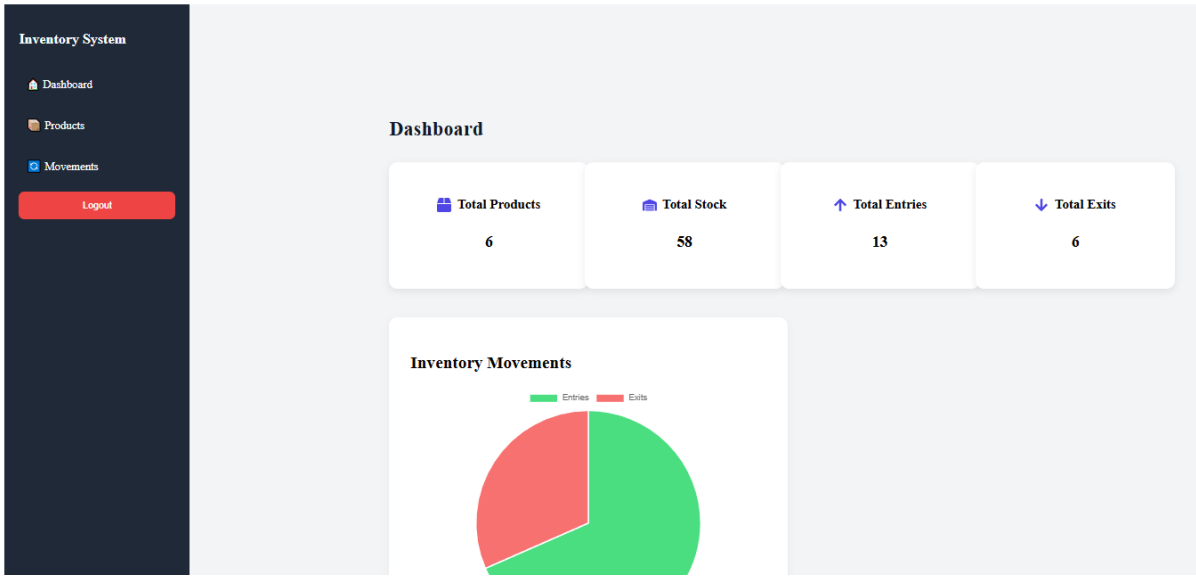
Inventory Management System

Versión 1.0

Autor: Diego Francisco Hernández Ramírez

Fullstack Project Documentation

Fecha: 19 de marzo del 2026



1. Project Overview

This project was developed as a personal challenge with the goal of learning and applying fullstack web development concepts. The main objective was to build a complete inventory management system from scratch, integrating both frontend and backend technologies.

Throughout the development process, I learned and implemented each technology progressively, solving real-world problems and improving my understanding of modern web development.

1. DESCRIPCIÓN GENERAL DEL PROYECTO

Este proyecto fue desarrollado como un desafío personal con el objetivo de aprender y aplicar conceptos de desarrollo web fullstack. El objetivo principal fue construir un sistema completo de gestión de inventario desde cero, integrando tecnologías frontend y backend.

A lo largo del proceso de desarrollo, aprendí e implementé cada tecnología progresivamente, resolviendo problemas del mundo real y mejorando mi comprensión del desarrollo web moderno.

2. Objectives

- Learn fullstack development (frontend + backend)
- Build a real-world application
- Understand API communication
- Implement authentication and security
- Work with databases and ORM tools
- Deploy a complete application

2. OBJETIVOS

- Aprenda el desarrollo fullstack (frontend + backend)
- Crea una aplicación del mundo real
- Comprender la comunicación API
- Implementar autenticación y seguridad
- Trabajar con bases de datos y herramientas ORM
- Implementar una aplicación completa

3. Technologies Used

Frontend:

- React
- Axios
- Chart.js

Backend:

- Node.js
- Express.js
- Prisma ORM

Database:

- PostgreSQL

Authentication:

- JSON Web Tokens (JWT)

3. TECNOLOGÍAS UTILIZADAS

Frontend:

- Reaccionar
- Axios
- Chart.js

Backend:

- Node.js
- Express.js
- Prisma ORM

Base de datos:

- PostgreSQL

Autenticación:

- Tokens web JSON (JWT)

4. Development Process

The project was developed step by step, starting with the backend structure, followed by database integration, and finally the frontend interface.

Key learning stages:

- Creating REST APIs with Express
- Managing data with Prisma and PostgreSQL
- Implementing authentication with JWT
- Connecting frontend and backend using Axios
- Handling errors and debugging deployment issues

This project helped reinforce problem-solving skills and the ability to learn new technologies independently.

4. PROCESO DE DESARROLLO

El proyecto se desarrolló paso a paso, comenzando con la estructura backend, seguido de la integración de la base de datos y finalmente la interfaz frontend.

Etapas clave de aprendizaje:

- Creación de API REST con Express
- Gestión de datos con Prisma y PostgreSQL
- Implementación de autenticación con JWT
- Conexión del frontend y el backend mediante Axios
- Manejo de errores y depuración de problemas de implementación

Este proyecto ayudó a reforzar las habilidades de resolución de problemas y la capacidad de aprender nuevas tecnologías de forma independiente.

5. System Features

- User authentication (login/register)
- Product CRUD operations
- Inventory tracking (entries and exits)
- Dashboard with analytics
- Low stock alerts
- Search, filters, and pagination
- CSV export functionality

5. CARACTERÍSTICAS DEL SISTEMA

- Autenticación de usuario (iniciar sesión/registrarse)
- Operaciones CRUD de productos
- Seguimiento de inventario (entradas y salidas)
- Panel de control con análisis
- Alertas de stock bajo
- Búsqueda, filtros y paginación
- Funcionalidad de exportación CSV

6. System Architecture

The system follows a client-server architecture:

Frontend (React): Handles user interface and interactions

Backend (Express): Processes requests and business logic

Database (PostgreSQL): Stores all application data

Communication is done through RESTful APIs.

6. ARQUITECTURA DEL SISTEMA

El sistema sigue una arquitectura cliente-servidor:

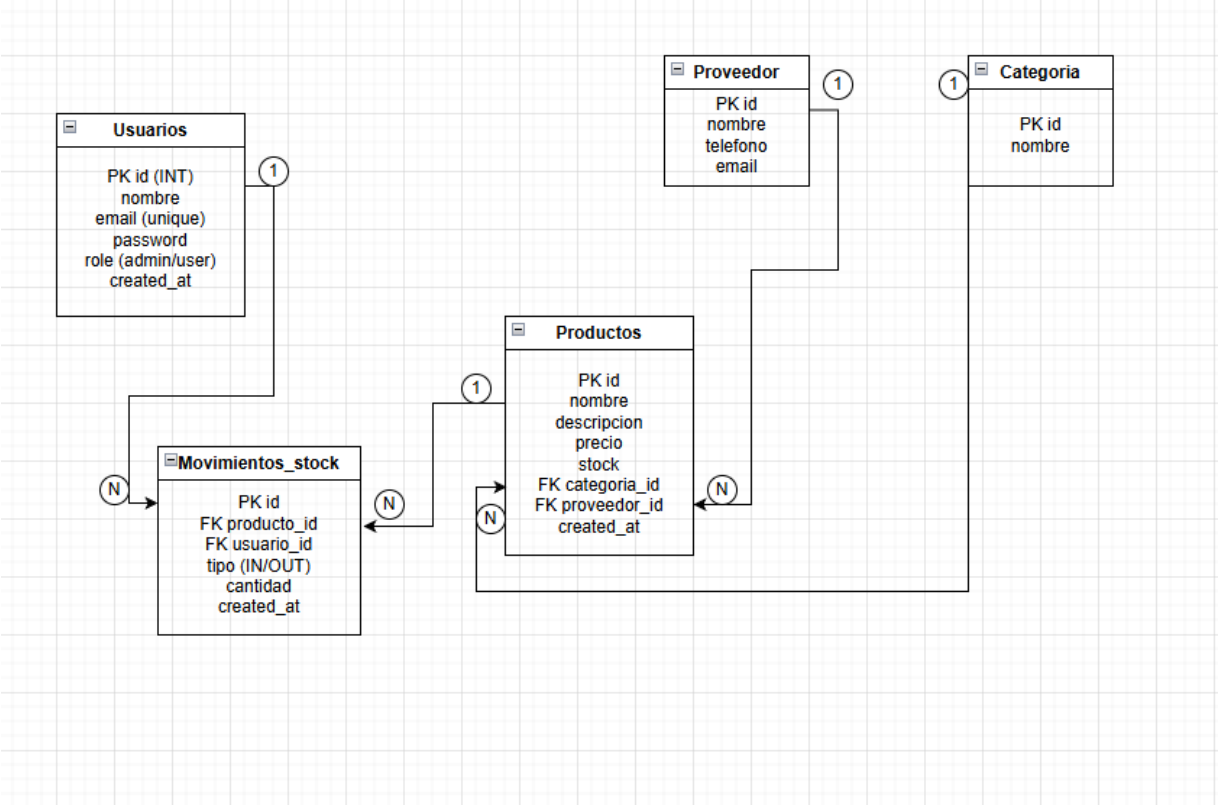
Frontend (React): maneja la interfaz de usuario y las interacciones

Backend (Express): Procesa solicitudes y lógica empresarial

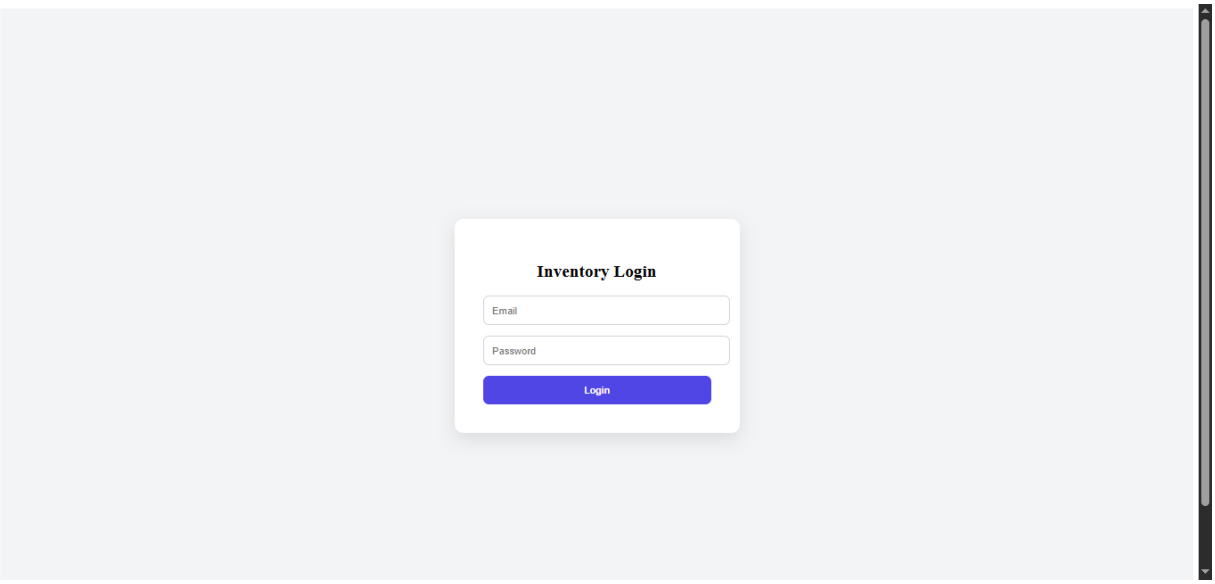
Base de datos (PostgreSQL): almacena todos los datos de la aplicación

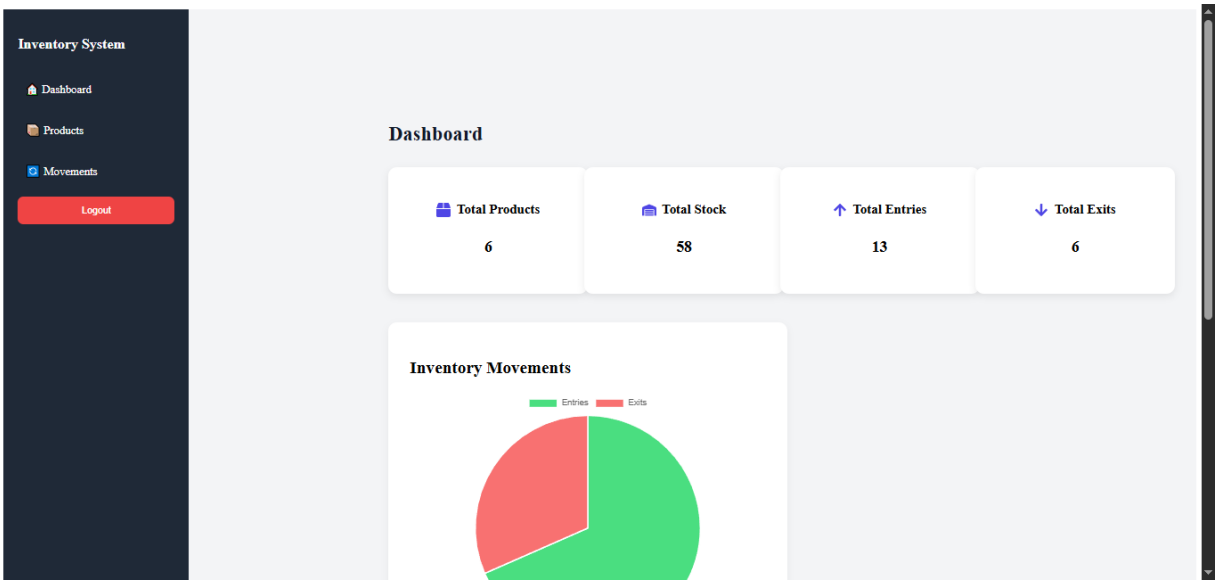
La comunicación se realiza a través de API RESTful.

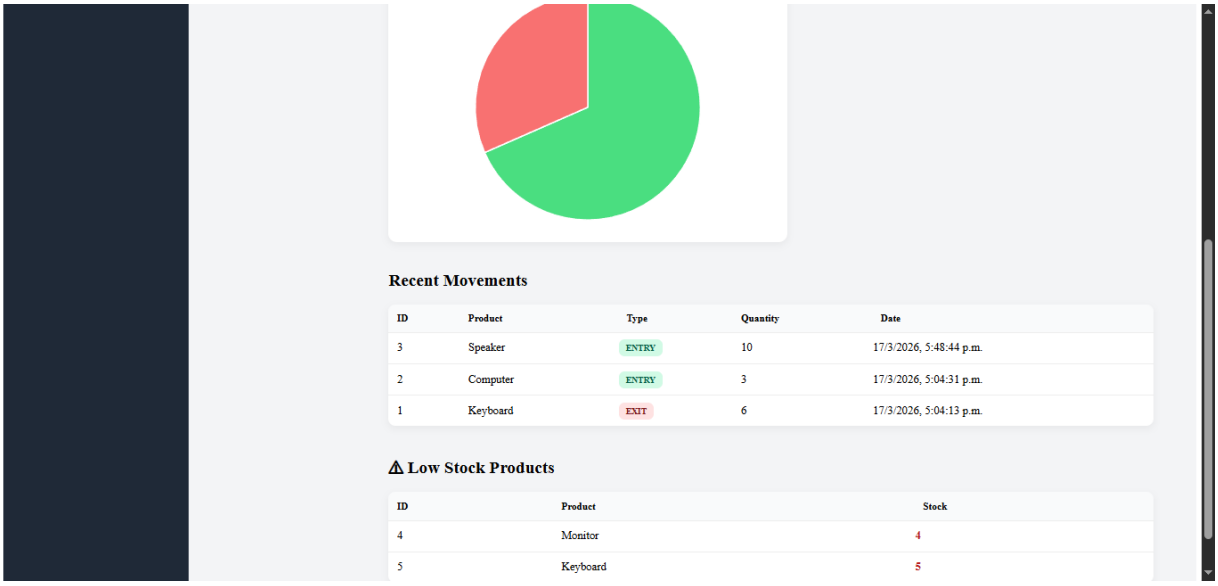
7. Diagram/Diagrama

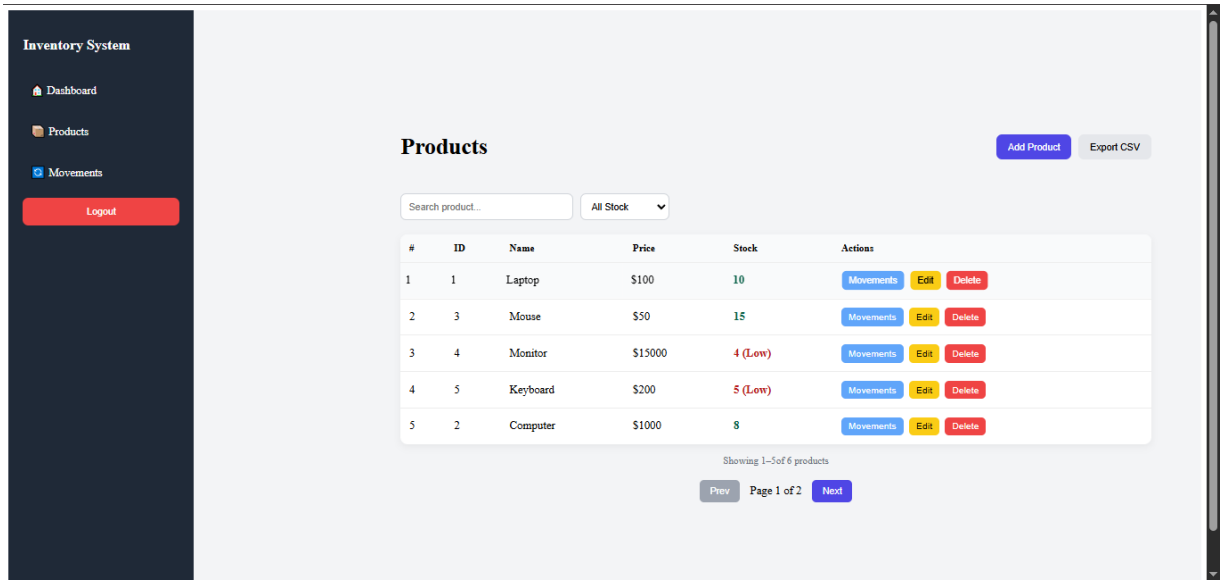


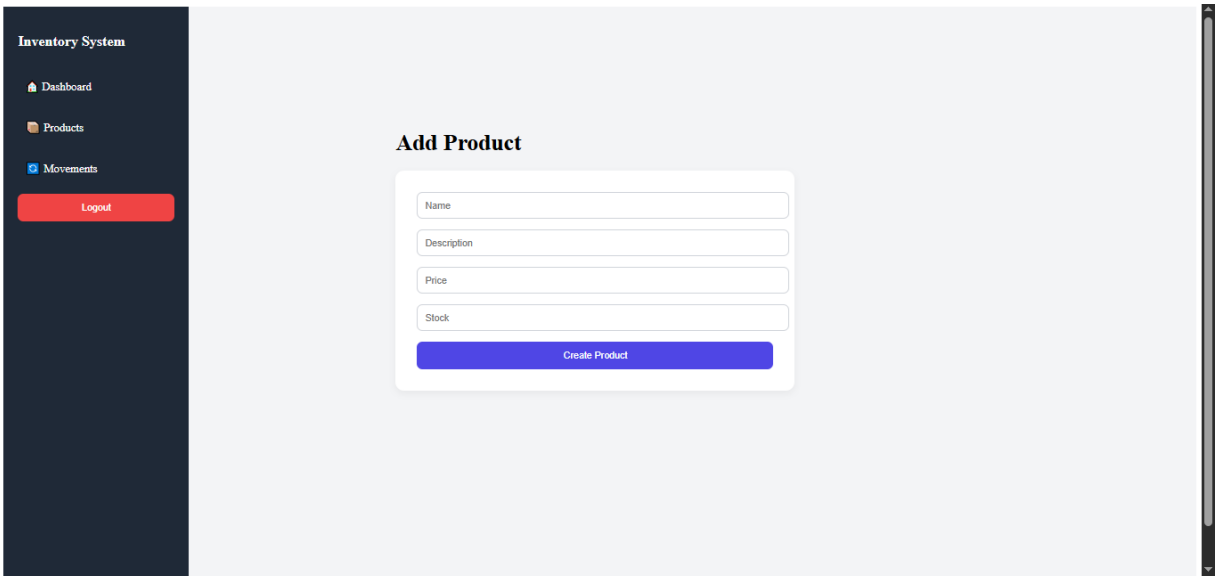
8. Images/Imágenes

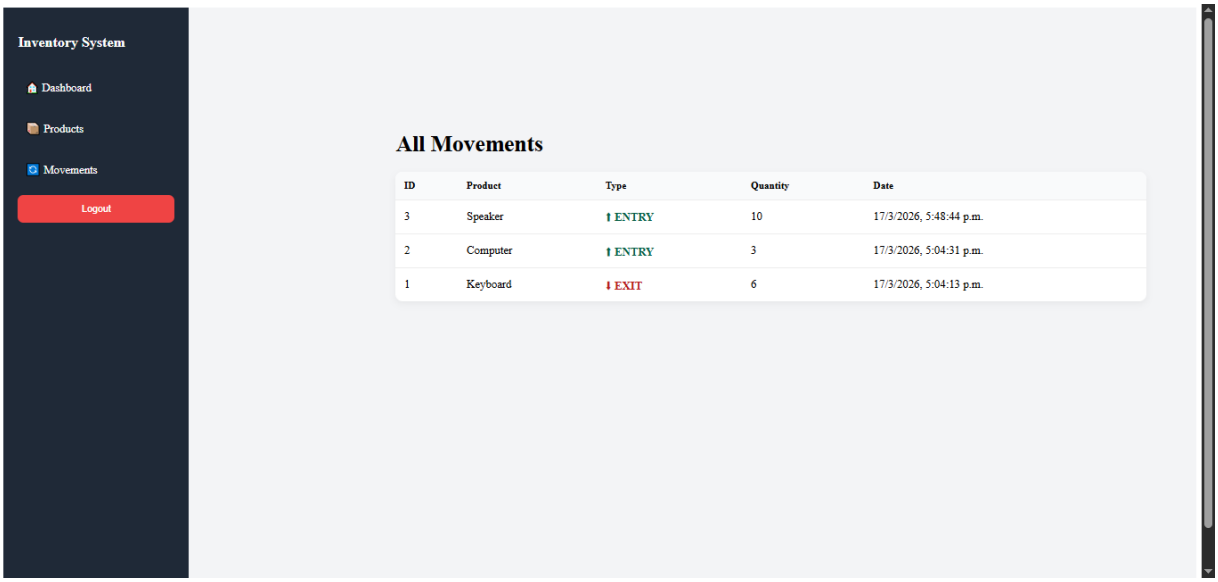





Recent Movements
ID Product Type Quantity Date
3 Speaker ENTRY 10 17/3/2026, 5:48:44 p.m.
2 Computer ENTRY 3 17/3/2026, 5:04:31 p.m.
1 Keyboard EXIT 6 17/3/2026, 5:04:13 p.m.
⚠ Low Stock Products
ID Product Stock
4 Monitor 4
5 Keyboard 5


Inventory System
Dashboard
Products
Movements
Logout
Products
Add Product
Export CSV
Search product...
All Stock
ID Name Price Stock Actions
1 1 Laptop $100 10 Movements Edit Delete
2 3 Mouse $50 15 Movements Edit Delete
3 4 Monitor $15000 4 (Low) Movements Edit Delete
4 5 Keyboard $200 5 (Low) Movements Edit Delete
5 2 Computer $1000 8 Movements Edit Delete
Showing 1–5of 6 products
Prev
Page 1 of 2
Next


Inventory System
Dashboard
Products
Movements
Logout
Add Product
Name
Description
Price
Stock
Create Product


Inventory System
Dashboard
Products
Movements
Logout
All Movements
ID Product Type Quantity Date
3 Speaker ↑ ENTRY 10 17/3/2026, 5:48:44 p.m.
2 Computer ↑ ENTRY 3 17/3/2026, 5:04:31 p.m.
1 Keyboard ↓ EXIT 6 17/3/2026, 5:04:13 p.m.

9. Authentication and Security

Authentication is handled using JWT tokens. After login, a token is generated and used to access protected routes.

Security features:

- Password hashing with bcrypt
- Protected API routes
- Role-based access (admin/user)

9. AUTENTICACIÓN Y SEGURIDAD

La autenticación se gestiona mediante tokens JWT. Después de iniciar sesión, se genera un token que se utiliza para acceder a rutas protegidas.

Características de seguridad:

- Hash de contraseñas con bcrypt
- Rutas API protegidas
- Acceso basado en roles (administrador/usuario)

10. Database Design

The database includes the following main entities:

- Users
- Products
- Movements

Relationships are managed using Prisma ORM, allowing efficient data handling and queries.

10. DISEÑO DE BASES DE DATOS

La base de datos incluye las siguientes entidades principales:

- Usuarios
- Productos
- Movimientos

Las relaciones se gestionan mediante Prisma ORM, lo que permite un manejo eficiente de datos y consultas.

11. Challenges and Solutions

During development, several challenges were encountered:

- Deployment issues (CORS, environment variables)
- Database synchronization
- Authentication errors

Each issue was resolved through debugging, documentation, and iterative testing.

11. DESAFÍOS Y SOLUCIONES

Durante el desarrollo se encontraron varios desafíos:

- Problemas de implementación (CORS, variables de entorno)
- Sincronización de bases de datos
- Errores de autenticación

Cada problema se resolvió mediante depuración, documentación y pruebas iterativas.

12. Future Improvements

- Improve UI/UX design
- Add reporting features
- Implement unit testing
- Optimize performance
- Add more user roles and permissions

12. MEJORAS FUTURAS

- Mejorar el diseño de UI/UX
- Agregar funciones de informes
- Implementar pruebas unitarias
- Optimizar el rendimiento
- Agregar más roles y permisos de usuario

13. Conclusion

This project represents a significant step in my journey as a developer. It demonstrates my ability to learn new technologies, solve problems, and build a complete fullstack application from scratch.

It also reflects my commitment to continuous learning and professional growth.

13. CONCLUSIÓN

Este proyecto representa un paso importante en mi trayectoria como desarrollador. Demuestra mi capacidad para aprender nuevas tecnologías, resolver problemas y crear una aplicación fullstack completa desde cero.

También refleja mi compromiso con el aprendizaje continuo y el crecimiento profesional.